

R1 Single-Arm Neural Reach, Grasp, and Throw

Teacher-guided residual learning with the Revo 2 Basic hand

Cesar A. Pascal Vasquez

Manual 3

Purpose of this Manual

This manual mounts the right BrainCo Revo 2 Basic hand on a fixed-pelvis Unitree R1 model and trains one neural policy to reach a randomized ball, close a stable grasp, lift and wind up, and release the ball forward. It explains the scripted teacher, observation and action spaces, residual network, behavioral cloning, DAGger correction, domain randomization, and held-out evaluation.

Experiment goal

Reproduce the saved 51-128-128-11 policy and evaluate the complete reach-grasp-throw task on 100 unseen randomized episodes in MuJoCo.

Contents

1	Before You Begin and Build the Model	3
2	System Integration	4
3	Task Timeline and Control Rate	5
4	Observations and Actions	6
5	How the Scripted Teacher Works	7
6	Neural Residual Policy	8
7	Choose Parameters and Write a Policy	9
8	Baseline Code Package and Data Flow	10
9	Model-Builder Code Walkthrough	11
10	Environment and Teacher Code Walkthrough	12
11	Neural and Training Code Walkthrough	13
12	Behavioral Cloning Stage	14
13	Dagger Correction Rounds	15
14	Domain Randomization for Robustness	16
15	Run, Evaluate, and Render	17
16	Recorded Results	18
17	Final Checklist and References	19

1 Before You Begin and Build the Model

This manual is self-contained. You need Python 3.10 or newer, MuJoCo 3, NumPy, the Unitree R1 MJCF with meshes, the BrainCo right Revo 2 XML with meshes, and the editable files under `editable_code/`. The exercise is simulation-only and does not send commands to an R1.

Create this local structure. Preserve every vendor mesh filename.

Minimum project tree

```
r1_single_arm_lab/  
|-- requirements.txt  
|-- scripts/build_model.py  
|-- scripts/r1_revo2_env.py  
|-- scripts/neural_policy.py  
|-- scripts/train_policy.py  
|-- scripts/play_policy.py  
|-- model/r1/r1_source.xml  
|-- model/r1/meshes/  
|-- model/revo2/brainco-righthand-v2.xml  
|-- model/revo2/meshes/  
|-- policies/  
|-- results/
```

Install, assemble, and inspect

```
cd "/path/to/r1_single_arm_lab"  
python3 -m venv .venv  
.venv/bin/python -m pip install "mujoco>=3.3.6,<4" "numpy>=2,<3"  
.venv/bin/python scripts/build_model.py  
.venv/bin/python scripts/probe_model.py
```

The build must create `model/r1_revo2_reach_throw.xml`. In the probe output, confirm 11 policy actuators and no original right-hand terminal geometry. Open the XML in a viewer before collecting data.

Checkpoint 1

Do not train until the assembled model loads, the right arm has five controlled joints, the Revo 2 hand has six controls, and the ball begins outside all collision meshes.

2 System Integration

The experiment uses the Unitree R1 body as the robot platform and replaces the original right terminal hand geometry with the BrainCo Revo 2 Basic model. A neutral visual sleeve and flange connect the wrist to the imported hand. The hand’s official meshes, joints, and actuator ranges are retained; the adapter is an approximation because a verified mechanical CAD interface was not available.

The pelvis is fixed to isolate upper-body control. The policy commands five right-arm actuators and six hand actuators, giving an 11-dimensional action. The XML is stored in `model/r1_revo2_reach_throw.xml`.



Figure 1: Fixed-pelvis R1 model with the stock right hand removed and the Revo 2 Basic hand mounted at the wrist.

Simulation boundary

This model validates a learning workflow in MuJoCo. The adapter, contact properties, actuator gains, and object dynamics must be identified on the physical robot before hardware deployment.

3 Task Timeline and Control Rate

The policy runs at 40 Hz: a new observation and action are computed every 0.025 s. MuJoCo performs multiple smaller physics steps between policy updates. One episode lasts 5.30 s and is divided into phases so the teacher and the loss can emphasize the behavior that matters at each time.

Table 1: Nominal single-arm episode

Time	Phase and required behavior
0.00–1.45 s	Move the right palm from its initial pose to the ball
1.48 s	Enable valid hand–ball contact evaluation
1.45–3.00 s	Close the Revo 2 fingers and establish the grasp
2.90 s	Release the initial soft fixture when the grasp is valid
3.00–3.45 s	Lift the ball while preserving contacts
3.45–3.80 s	Move the arm to the windup pose
3.88 s	Open the hand and begin free ball flight
3.88–4.30 s	Follow through toward the forward target
4.30–5.30 s	Observe flight and calculate final metrics

At reset, a random delay of zero to two control ticks can be applied. This prevents a policy from relying on one exact timing alignment.

4 Observations and Actions

The neural policy receives 51 values. Every quantity is normalized to a useful numerical range before it reaches the network.

Table 2: The 51-dimensional observation vector

Group	Size	Included information
Task context	13	Normalized time and phase, ball relative to palm, target relative to ball
Robot and object state	27	Palm and ball motion, arm state, hand ratios, contacts
Action memory	11	Previous arm and hand command
Total	51	Complete input supplied at each 40 Hz update

The action has 11 normalized values: five for the arm and six for the Revo 2 hand. The learned network does not replace the teacher trajectory completely. It predicts a residual correction r around the teacher command a_T :

$$a = \text{clip}(a_T + s \odot \tanh(r), a_{\min}, a_{\max}).$$

The scale is ± 0.25 for arm channels and ± 0.08 for hand channels. These bounds let learning correct randomized errors while keeping early rollouts near the demonstrated motion.

5 How the Scripted Teacher Works

The teacher is a deterministic program, not a second neural network. It knows the simulated ball pose and the task clock. For each phase, it selects a desired palm waypoint and uses inverse kinematics to calculate arm commands. A timed hand schedule opens, closes, holds, and releases the six finger controls.

1. **Reach:** generate a palm target slightly behind the randomized ball.
2. **Grasp:** interpolate toward the power-grasp ratios while maintaining palm alignment.
3. **Lift:** move the target upward only after the contact envelope is valid.
4. **Windup:** move the arm backward to store forward throwing motion.
5. **Release:** open the hand at the scheduled release time and follow through.

Randomized waypoints and object properties produce varied demonstrations. The data-generation run collected 240 teacher episodes; 88.3% completed the full success condition. Teacher failures remain useful only when state-action pairs are valid, but the main dataset is dominated by successful and near-successful trajectories.

Why use a teacher?

Pure random exploration almost never discovers the ordered sequence of reaching, grasping, lifting, and releasing. The scripted teacher places the policy near useful states; learning then makes those actions responsive to observations and randomization.

6 Neural Residual Policy

The policy is a multilayer perceptron with 51 inputs, two hidden layers of 128 tanh units, and 11 outputs. The final outputs pass through tanh and become bounded residuals. The network therefore contains

$$(51 \times 128 + 128) + (128 \times 128 + 128) + (128 \times 11 + 11) = 24,587$$

trainable weights and biases.

```
1 def forward(obs):  
2     x = normalize(obs)  
3     h1 = np.tanh(x @ W1 + b1)  
4     h2 = np.tanh(h1 @ W2 + b2)  
5     residual = np.tanh(h2 @ W3 + b3)  
6     return clip(teacher_action(obs) + scale * residual)
```

Listing 1: Conceptual forward pass

Tanh hidden units are smooth and bounded, which is helpful for a small continuous-control model. The saved weights are in `policies/r1_revo2_reach_grasp_throw.npz`. During playback, the program loads the normalization statistics and weights, evaluates one forward pass every 0.025 s, and sends the resulting 11 controls to MuJoCo.

7 Choose Parameters and Write a Policy

Start with a smoke test, then select a training budget that answers one question. The ranges below are useful simulation-study ranges; they are not hardware commands.

Table 3: Editable single-arm training parameters

Parameter	Reproduction	Study range	Effect
Hidden units/layer	128	64–256	Network capacity and cost
Teacher episodes	240	40–400	Demonstration diversity
Initial updates	3,600	500–8,000	Behavioral-cloning fit
DAgger rounds	2	0–4	Off-trajectory recovery data
DAgger episodes	60	20–120	New states collected per round
DAgger updates	1,800	500–4,000	Fit to each aggregated dataset
Batch size	512	128–1,024	Gradient noise and memory
Learning rate	1.2×10^{-3}	3×10^{-4} – 2×10^{-3}	Convergence speed and stability

To write your own controller, implement `policy(observation)->action`, where the input shape is 51 and output shape is 11. Keep residual bounds during early experiments. The reference policy exposes `nominal_actions()`, so a new head can use the same teacher trajectory:

```
1 class StudentResidual:
2     def __init__(self, reference, W, b):
3         self.reference, self.W, self.b = reference, W, b
4
5     def __call__(self, obs):
6         obs = np.asarray(obs, dtype=float)
7         nominal = self.reference.nominal_actions(obs[None, :])[0]
8         correction = np.tanh(self.W @ obs + self.b)
9         scale = np.r_[np.full(5, .25), np.full(6, .08)]
10        return np.clip(nominal + scale * correction, -1, 1)
```

Listing 2: Minimal custom residual head

Design before training

Hypothesis: _____

Change: _____ Fixed seed blocks: training _____, evaluation _____.

8 Baseline Code Package and Data Flow

The complete baseline source is in `editable_code/`. Copy `requirements.txt` to the project root and the contents of `editable_code/scripts/` to the local `scripts/` folder.

Table 4: Single-arm baseline files and reading order

Order	File	Responsibility
1	<code>build_model.py</code>	Combines the R1 and right-hand XML assets
2	<code>probe_model.py</code>	Loads the result and checks bodies, joints, actuators, and sites
3	<code>r1_revo2_env.py</code>	Defines task timing, observations, teacher, randomization, reward, and metrics
4	<code>neural_policy.py</code>	Defines normalization, residual MLP, loss, Adam, and NPZ save/load
5	<code>train_policy.py</code>	Collects teacher data, clones, runs DAgger, evaluates, and writes results
6	<code>play_policy.py</code>	Opens the trained simulation with selected episodes and seeds
7	<code>evaluate_policy.py</code>	Runs held-out episodes and saves JSON
8	<code>render_policy.py</code>	Produces trained, untrained, or teacher MP4 output

The baseline data path is **XML assets** → **environment** → **teacher demonstrations** → **normalized observations** → **neural residuals** → **MuJoCo actions** → **metrics**. Training and evaluation instantiate the same environment but use different seed blocks.

Keep an unchanged reference

Copy the full project before editing. A model-building change can invalidate an old policy even when the Python network is unchanged. Save the assembled XML, policy NPZ, training JSON, and exact command together as one reproducible experiment record.

9 Model-Builder Code Walkthrough

Open `build_model.py`. The first constants are the three paths students must verify: `R1_SOURCE`, `HAND_SOURCE`, and `OUTPUT`. All other paths are written relative to the final XML.

Table 5: Where R1–Revo integration is implemented

Search for	What the function controls
<code>rewrite_mesh_paths</code>	Prefixes R1 and hand mesh references after XML merging
<code>joint_gains</code>	Selects R1 position gain, velocity gain, and force limit by joint name
<code>add_r1_actuators</code>	Creates actuators for the R1 joints retained in the model
<code>add_scene</code>	Adds floor, lights, ball, target markers, and soft fixture
<code>build</code>	Loads both trees, removes floating base and stock hand, mounts Revo 2, and writes output

Inside `build()`, the MuJoCo option block is another controlled parameter location.

```
1 "timestep": "0.0025",
2 "integrator": "implicitfast",
3 "iterations": "24",
4 "ls_iterations": "8",
5 "gravity": "0 0 -9.81",
```

Listing 3: Physics parameters created by the baseline builder

Change integration settings only after the unchanged model passes `probe_model.py`. Smaller time steps improve contact resolution but increase runtime. Any wrist transform or adapter change requires rebuilding the XML and visually checking mesh overlap before training.

Rebuild after a model edit

```
.venv/bin/python scripts/build_model.py
.venv/bin/python scripts/probe_model.py
```

10 Environment and Teacher Code Walkthrough

Open `r1_revo2_env.py`. Search for class `TaskConfig`; this is the timing and sensor-noise control panel. The teacher and randomization are in the same file so a reader can trace every demonstration back to simulation state.

Table 6: Exact environment edit locations

Search for	Parameters or behavior at that location
<code>TaskConfig</code>	40 Hz rate, phase times, delay maximum, observation noise
<code>_restore_and_randomize_model</code>	Gain 0.82–1.18, radius 0.0165–0.0195 m, mass 0.038–0.070 kg, friction 0.75–1.45
<code>_compute_waypoints</code>	Randomized reach, lift, windup, and throw targets
<code>solve_ik</code>	IK iterations, damping, and joint solution
<code>teacher_action</code>	Phase-dependent arm waypoints and six hand commands
<code>_observation</code>	The 51 values and their normalization/order
<code>_update_events</code>	Contact gate and fixture-release logic
<code>_reward</code>	Dense learning score
<code>metrics</code>	Reach, grasp, hold, release, speed, distance, and success thresholds

For an easier first experiment, reduce randomization but do not remove it silently. For example, narrow ball mass to 0.045–0.060 kg inside `_restore_and_randomize_model()`. Train and evaluate a second policy with the original range, using the same held-out seed block.

Observation changes require policy changes

If a feature is added or removed in `_observation()`, update `observation_size`, saved normalization data, the first network layer, and every trained NPZ. Old policies cannot be assumed compatible.

11 Neural and Training Code Walkthrough

Open `neural_policy.py`. The constructor sets hidden width, initialization seed, arm residual scale, and hand residual scale. `_forward()` contains two tanh hidden layers; `nominal_actions()` reproduces the central scripted trajectory; `decode_actions()` adds the bounded residual; and `train_batch()` implements weighted mean-squared error and Adam.

Open `train_policy.py`. Its four stages are visible in `main()`: collect teacher data, run initial cloning, collect and train two DAgger rounds, then evaluate on seeds beginning at 53000.

Table 7: Parameters: terminal flag or source edit

Parameter	Change at	Notes
Hidden size	<code>--hidden-size</code>	Rebuilds all three weight matrices
Teacher/Dagger budget	flags in <code>train_policy.py</code>	No source edit required
Batch/LR	<code>--batch-size/--learning-rate</code>	Stored in training metadata
Policy filename	<code>--output</code>	Keeps baseline and edited NPZ files separate
Residual limits	<code>NeuralPolicy.__init__</code>	Defaults 0.25 arm and 0.08 hand
Arm/hand loss weights	<code>output_weights</code>	Defaults 4.0 and 1.5
Phase balancing	<code>phase_buckets</code>	Six phases sampled uniformly
Teacher mixing	<code>dagger_probabilities</code>	Decreases 0.45 to 0.15
Seed blocks	<code>first_seed</code> calls	23000 teacher, 33000 DAgger, 53000 evaluation

Baseline and one controlled modification

```
.venv/bin/python scripts/train_policy.py \
--hidden-size 128 --initial-steps 3600 \
--dagger-steps 1800 \
--output policies/baseline_h128.npz
.venv/bin/python scripts/train_policy.py \
--hidden-size 192 --initial-steps 3600 \
--dagger-steps 1800 \
--output policies/edited_h192.npz
.venv/bin/python scripts/evaluate_policy.py \
--policy policies/baseline_h128.npz \
--episodes 100 --first-seed 63000 \
--output results/baseline_h128_eval.json
.venv/bin/python scripts/evaluate_policy.py \
--policy policies/edited_h192.npz \
--episodes 100 --first-seed 63000 \
--output results/edited_h192_eval.json
```

The same seed block makes this a controlled architecture experiment. The supplied baseline accepts `--output` in training and `--policy` in playback/evaluation so student runs do not overwrite one another.

12 Behavioral Cloning Stage

Behavioral cloning begins with the teacher dataset. For each teacher state o_i , the target is the residual required to reproduce the teacher command. Adam minimizes a weighted mean-squared error:

$$L(\theta) = \frac{1}{N} \sum_i \|W(\pi_\theta(o_i) - a_i^*)\|_2^2.$$

Arm outputs use weight 4.0 and hand outputs use weight 1.5. The larger arm weight emphasizes palm placement, while the hand weight still preserves the grasp schedule. Batches are balanced across six task phases so the long reach portion cannot overwhelm the short release phase.

The exact saved-policy reproduction uses 3,600 initial Adam updates, batch size 512, hidden size 128, and initial learning rate 1.2×10^{-3} . Observation jitter exposes the network to small state errors around each teacher trajectory.

Start the full training run

```
.venv/bin/python scripts/train_policy.py \  
--initial-steps 3600 --dagger-steps 1800
```

Behavioral cloning is fast, but it sees mainly teacher states. Small prediction errors can move the robot into states missing from the original dataset, which motivates DAgger.

13 DAgger Correction Rounds

Dataset Aggregation, or DAgger, alternates between policy rollouts and teacher relabeling. The current neural policy runs in MuJoCo, and the scripted teacher provides the action it would choose at each visited state. Those new pairs are appended to the training set.

1. Train on the initial teacher demonstrations.
2. Run the neural policy under randomized conditions.
3. Query the teacher at states reached by that policy.
4. Add the new labeled states to the dataset.
5. Retrain and repeat for a second round.

The saved run used two DAgger rounds, 60 rollout episodes per round, and 1,800 optimizer updates after each aggregation. The complete run therefore used 7,200 optimizer updates and ended with 63,985 state–action samples.

Expected result

DAgger does not invent a new objective. It teaches the same demonstrated behavior at the off-trajectory states caused by the policy’s own errors, delay, noise, and randomized physics.

14 Domain Randomization for Robustness

Each episode changes simulation conditions within bounded ranges. The policy must infer corrective behavior from its current observation instead of memorizing one trajectory.

Table 8: Training randomization

Parameter	Range or variation
Ball start	Approximately ± 20 mm around the nominal position
Ball radius	16.5–19.5 mm
Ball mass	38–70 g
Contact friction	0.75–1.45 scale range
Actuator gain	82–118% of nominal
Control delay	0–2 policy ticks
Observation noise	Gaussian noise with scale near 0.010
Throw target	Small position variation around the nominal target

Randomization should remain physically plausible. Excessively wide ranges can turn training into a different task or hide integration errors. Evaluation must use unseen seeds, not the demonstration or DAgger seeds.

15 Run, Evaluate, and Render

After the environment has been created once, the commands below reproduce the final policy behavior. Replace the example path with the location of the project.

Interactive final result

```
cd "/path/to/r1_single_arm_lab"  
.venv/bin/python scripts/play_policy.py --episodes 3
```

Held-out evaluation

```
.venv/bin/python scripts/evaluate_policy.py --episodes 100
```

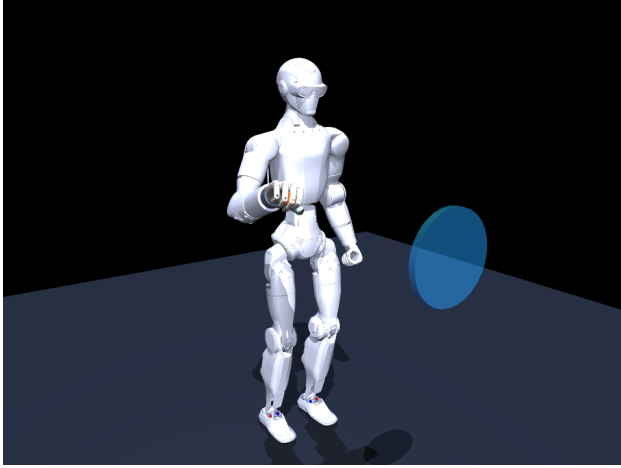
Render trained and untrained videos

```
.venv/bin/python scripts/render_policy.py --seed 53000  
.venv/bin/python scripts/render_policy.py \  
  --mode untrained --seed 53000
```

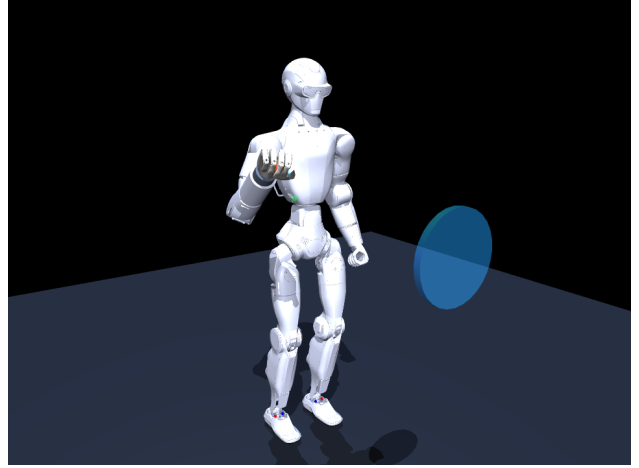
Use the same task seed for the trained and untrained renders. The only intended difference is the policy weights, which makes the visual comparison meaningful.

16 Recorded Results

The held-out evaluation used 100 unseen randomized episodes. The complete-task success rate was 81%. Reach succeeded in every episode, with 13.4 mm mean minimum reach error. Grasp succeeded in 97%, while hold-and-release succeeded in 88%. Mean peak forward ball speed was 0.58 m/s, and mean forward travel was 0.20 m.



Neural policy during grasp.



Neural policy during release.

Figure 2: Two stages from the trained single-arm rollout.

These metrics show robust simulation performance across the chosen distributions, not guaranteed transfer to the physical R1. The remaining failure concentration around hold and release suggests improving wrist alignment, contact identification, or the release-time representation before hardware testing.

17 Final Checklist and References

- ☐ The stock right hand geometry is absent from the assembled XML.
- ☐ The observation length is 51 and the action length is 11.
- ☐ Training metadata records 240 teacher episodes and two DAgger rounds.
- ☐ Evaluation uses 100 seeds excluded from training.
- ☐ Reach, grasp, release, speed, and distance are reported separately.
- ☐ Simulation assumptions are disclosed before hardware transfer.

References

1. BrainCo Hand. (n.d.). *Revo 2 parameters*.
<https://www.brainco-hz.com/docs/revolimb-hand-dev/en/revo2/parameters.html>
2. DeepMind. (2026). *MuJoCo XML reference*.
<https://mujoco.readthedocs.io/en/stable/XMLreference.html>
3. Ross, S., Gordon, G., and Bagnell, D. (2011). A reduction of imitation learning and structured prediction to no-regret online learning. *Proceedings of AISTATS*, 15, 627–635.
<https://proceedings.mlr.press/v15/ross11a.html>
4. Unitree Robotics. (n.d.). *R1 humanoid robot*.
<https://www.unitree.com/R1>